
ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTY OF ELECTRICAL AND COMPUTER ENGINEERING
SOFTWARE ENGINEERING II
CONTINUOUS INTEGRATION WORKSHOP - II TERM 2025

Objectives:

- Automatically run tests after uploading new code to an existing repository using Jenkins and Github Actions
- Obtain a report with the results of the tests.

Requirements:

- Jenkins
- Ngrok
- A programming language IDE, like Eclipse or Visual Studio Code.
- Git

Introduction:

Continuous integration is a software development practice where new code is integrated into existing code hosted in a repository. This integration triggers automatic execution of tests to ensure its correct operation [1].

The goal of continuous integration is to quickly detect and locate bugs, so reduce the time to fix and improve the software, and that updates are available as soon as possible.

To carry out this workshop, you must use Jenkins as a continuous integration tool. Jenkins is Open Source and runs as a server on the host machine; In addition, it has numerous plugins that increase its capabilities.

Alternatively, GitHub Actions provides another approach. It's a powerful tool integrated with GitHub repositories that automates workflows such as testing, building, and deploying software projects, offering flexibility in managing your CI/CD pipelines.

Additionally, you will use Ngrok, which is a tool that allows exposing a local url to a public url.

For this workshop, Jenkins will run locally on your computer, by uploading the changes to your remote repository hosted on Github, it will be able to send a response to our server, so it is necessary to have a public url to expose our local server to the internet. Here Ngrok intervenes that will act as a communication bridge between Github and Jenkins.

Activities

Create a command-line application and use diverse tools to successfully integrate CI in your repository.

For this workshop, we will explore two different CI tools—**Jenkins** and **GitHub Actions**—and highlight their differences and strengths. You must work through both sections, as each tool has a different objective and offers unique benefits for CI/CD.

- **Jenkins:** We will focus on setting up automatic test execution for each push to the master branch. Jenkins is a powerful and customizable tool, well-suited for more complex build and test scenarios, especially when dealing with multiple systems.
- **GitHub Actions:** We will develop a workflow that runs for each pull request targeting the master branch, with a focus on linting the code. GitHub Actions integrates seamlessly with GitHub, making it a convenient choice for repository-related tasks like code linting.

Using both Jenkins and GitHub Actions in the same project allows you to leverage their individual strengths—Jenkins for sophisticated automation across environments and GitHub Actions for rapid, GitHub-centric workflows. Combining these tools can create a more comprehensive CI/CD pipeline, providing flexibility and ensuring code quality at multiple stages.

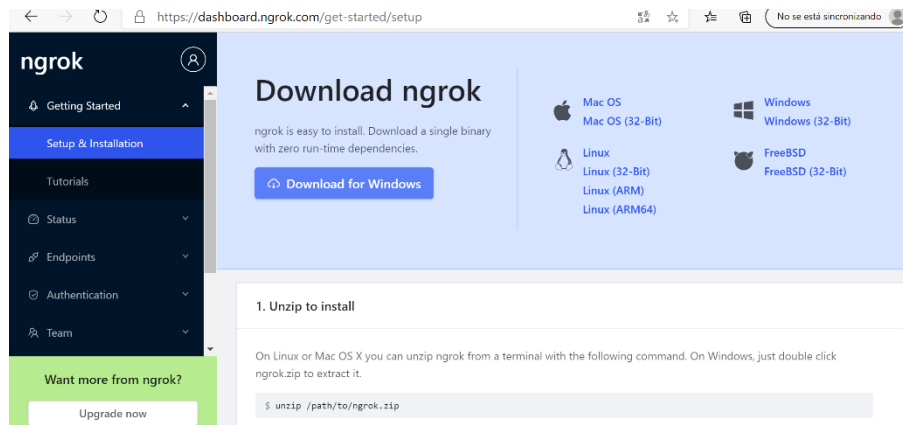
Section A – Jenkins Configuration

Step 1: Install Requirements

1. Install Jenkins
 - a. Go to the following <https://jenkins.io/download/>
 - b. Download Jenkins (LTS o Weekly) for your OS.
 - c. Follow the steps shown in the following tutorial:

Windows [2] o ubuntu [3] or Docker
 - d. Go to your jenkins local and login with the admin password. Follow the steps and install all the suggested plugins.
2. Install ngrok
 - a. Go to the following <https://dashboard.ngrok.com/signup>
 - b. Create an account. Github is recommended.

- c. Download the file.



- d. Unzip the file.
- e. Open the program and enter the following command
Note: The authtoken is in the 2nd item of the ngrok page.

```
//Replace <your authtoken> with the token given in the page
./ngrok authtoken <your authtoken>
```

2. Connect your account

Running this command will add your authtoken to the default `ngrok.yml` configuration file. This will grant you access to more features and longer session times. Running tunnels will be listed on the [endpoints](#) page of the dashboard.

```
$ ngrok config add-authtoken <your authtoken>
```

- f. To test the operation, write the following command.

```
./ngrok http 8080
```

```
ngrok
( Ctrl+C to quit)

? Try the ngrok Kubernetes Ingress Controller: https://ngrok.com/s/k8s-ingress

Session Status      online
Account             NataliaMw (Plan: Free)
Version             3.3.1
Region              South America (sa)
Latency             145ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://1c55-181-199-60-201.ngrok-free.app -> http://localhost:8080

Connections
  ttl    opn    rt1    rt5    p50    p90
   47     1   0.46   0.13   0.43   0.91

HTTP Requests
-----
```

- g. Take note of the highlighted URL and do not close the console.

Note: ngrok will give you a random URL every time the command is entered. Do not close.

Step 2: Create a Repository

1. Create a Github repository.
2. Set up your Jenkins connection:
 - a. Go to Settings > Webhook > Add Webhook
 - b. In Payload URL, place your url as follow:
<your Jenkins url>/github-webhook/
For example: `http://e760c763.ngrok.io/github-webhook/`
 - c. In Content type, select application/json
 - d. Leave the rest of the options as they are and click on Add WebHook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

SSL verification
 By default, we verify SSL certificates when delivering payloads.
☒ **Enable SSL verification** ☐ Disable (not recommended)

Which events would you like to trigger this webhook?
☒ Just the push event.
☐ Send me everything.
☐ Let me select individual events.

☒ **Active**
 We will deliver event details when this hook is triggered.

Add webhook

- e. Check the console again and verify the connection.

```

HTTP Requests
-----
POST /github-webhook/ 200 OK
  
```

Step 3: Prepare Jenkins

1. Select option "Manage Jenkins" > Manage Plugins
2. Select the Available tab and search "test-results-analyzer"
3. Mark the check Install y click "Install without restart"

Step 4: Create a project in Jenkins

1. Choose a programming language of your preference where you will develop the project.
2. Open Jenkins.
3. Select the option "New Item" in the left menu.
4. Enter a name, select "Freestyle Project", and click Ok.
5. In the tab "General", mark Github Project and write your repository URL.

☐ Desechar ejecuciones antiguas ?

☐ Esta ejecución debe parametrizarse ?

☒ **GitHub project**

Project url ?

6. In the tab "Source Code Management", select git and enter the url again.

Repositories ?

Repository URL ?

<https://github.com/NataliaMw/ContinuousIntegration>

7. Check the branch name and rename it if needed.

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Navegador del repositorio ?

8. In the tab “Build Trigger”, select the option “GitHub hook trigger for GITScm polling.”

9. In the tab “Build Steps”

- Select “Execute Windows batch command.”
- In the text area, write line by line the necessary steps for execute the tests in the selected language.

For example, in python it should be something like this:

Build Steps

Ejecutar un comando de Windows ?

Comando

Visualiza [la lista de variables de entorno disponibles](#)

```
python -m venv env
call env\Scripts\activate
pip install pytest
pytest --junitxml=test-results.xml
deactivate
```

Avanzado ▾

Añadir un nuevo paso ▾

Note: Understanding Jenkins Build Commands and Python Errors

The commands entered in the “**Execute Windows batch command**” section of the **Build Steps** run **inside the environment where the Jenkins server is installed**—in this case, your local machine or the Jenkins agent (if one is used).

If you receive an error like:

'python' is not recognized as an internal or external command

this means **Jenkins cannot find Python in the system's PATH**. Even if Python is installed on your machine, Jenkins may not be able to access it due to:

- Jenkins running as a **Windows service** under a different user account (which doesn't have Python in its PATH).
- Python not being added to the system PATH during installation.

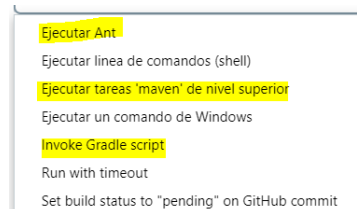
To fix this:

- Make sure Python is installed and working from your regular command line.
- Add Python to the **system environment variables (PATH)** if it wasn't added during installation.

3. If Jenkins is running as a service:
 - o Open **Services**, find Jenkins, right-click → **Properties**.
 - o In the **Log On** tab, switch to a user account that has access to Python.
 - o Restart the Jenkins service after making this change.
4. Alternatively, use the **full path to the Python executable** in the Jenkins build step (e.g., `C:\Python311\python.exe script.py`) instead of just `python`.

This applies similarly for other command-line tools like `npm`, `pytest`, or `node`.

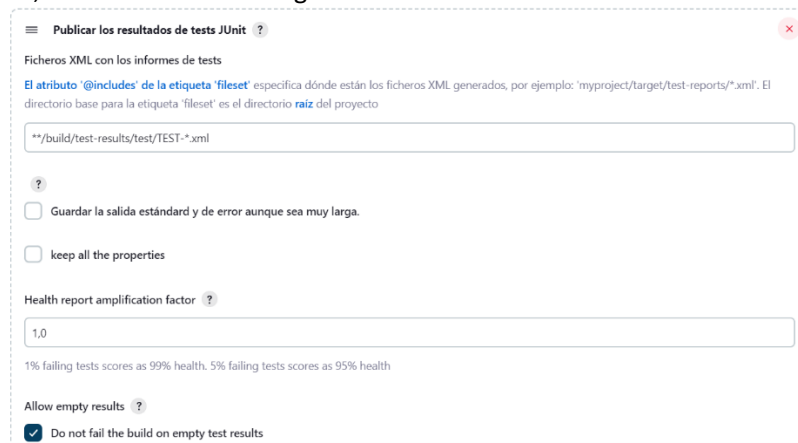
- c. If the selected language is Java, it has already integrated tools, so you don't need to use the command line:



- d. In the command line, when executing the tests, add an extra line to export them in xml. For example, it should be like this for python or javascript:
 - i. `pytest --junitxml=test-results.xml`
 - ii. `npx jest --ci --testResultsProcessor="jest-junit"`

10. In the tab "Post-build Actions", enter data as follow:

- a. Select Publish Junit test result report.
- b. In the input, add the route where the tests will be generated after the tests. That was the configuration in step 9.d.
For java, it should be something like this:



- c. Check the Do not fail the build on empty test results.

11. Click Save.

Section B – Github Actions Configuration

Step 1: Configure the github repository

1. Use the previous created Github repository.
2. Create a workflow file `.github/workflows/ci.yml`.
3. Or: Go to the section "Actions" and select set up a workflow yourself:

Get started with GitHub Actions

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started.

Skip this and [set up a workflow yourself](#) →

Q Search workflows

Suggested for this repository

Docker image
 By GitHub Actions
 Build a Docker image to deploy, run, or push to a registry.
[Configure](#)


Python Package using Anaconda
 By GitHub Actions
 Create and test a Python package on multiple Python versions using Anaconda for package management.
[Configure](#)


Jekyll using Docker image
 By GitHub Actions
 Package a Jekyll site using the jekyll/bx Docker image.
[Configure](#)

Build projects with Make
 By GitHub Actions
 Build and test a project using Make.
[Configure](#)


Publish Python Package
 By GitHub Actions
 Publish a Python Package to PyPI on release.
[Configure](#)


Django
 By GitHub Actions
 Build and Test a Django Project
[Configure](#)

- Define workflow steps using YAML syntax to automate testing and build processes. The next steps will show examples of how to customize your workflow according to the needs of this workshop.

Step 2: Customize When Workflow Runs Are Triggered

In this step, you will define when your workflow should be triggered. Below are three common options you can choose from:

Note: Take in account that these are only snippets examples, it is not at all the complete workflows. Merely some of the configuration options available.

Option 1: Trigger Workflow on Push Events to Specific Branches

You can set your workflow to run when code is pushed to the main branch or any branch under release/*.

```

on:
  push:
    branches:
      - main
      - release/*
  
```

Option 2: Trigger Workflow on Pull Requests Targeting the Main Branch

You can set your workflow to run when a pull request targets the main branch. This is often useful for ensuring changes are properly vetted before merging.

```

on:
  pull_request:
    branches:
      - main
  
```

Option 3: Trigger Workflow on a Schedule

You can schedule your workflow to run automatically every weekday (Monday to Friday) at 02:00 UTC.

```
on:
  schedule:
    - cron: "0 2 * * 1-5"
```

Recommendation

For this workshop, it is recommended to use **Option 2**: triggering on pull requests to the main branch, as it aligns best with the workshop's objectives and ensures changes are reviewed before being merged.

Step 3: Using an action

Actions are reusable units of code that can be built and distributed by anyone on GitHub. You can find a variety of actions in [GitHub Marketplace](#), and in the official [Actions repository](#).

To use an action, you must specify the repository that contains the action. We also recommend that you specify a Git tag to ensure you are using a released version of the action.

```
- name: Setup Node
  uses: actions/setup-node@v4
  with:
    node-version: '20.x'
```

Step 4: Running a command

You can run commands using this syntax:

```
- name: Install Dependencies
  run: npm install

- name: Analyzing the code with pylint
  run: |
    pylint $(git ls-files '*.py')
```

You will need to create a pipeline for every open pull request to the main branch, that will automatically run some linting tool, like the ones checked in the Coding Standards workshop.

Section C - Development

Step 5: Develop a program with the given requirements.

1. Read carefully the requirements given in the last part of this document.
2. Start a new project in the repository created before.
3. Implement the requirements provided at the end of this document.
 - a. For this part of the workshop, you will be required to work in group. Each member should assign themselves a part of the requirements to code and will open branches to the main branch whenever you need to merge your code.
4. Add a tool for testing if it needs to be installed. For example, Junit, Pytest, Jest, etc.
5. Develop unit tests corresponding to the implemented code to verify the fulfillment of the requirements.

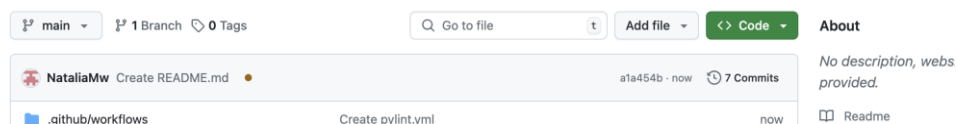
6. Do not forget to add the linting tool that will be triggered for Github Actions.

Step 6: Verify CI Integration

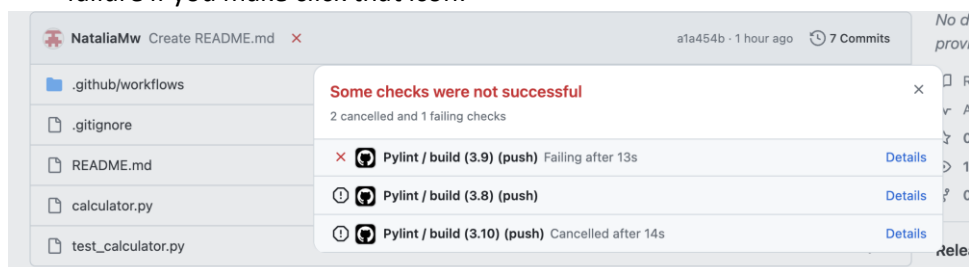
1. Push to the repository the developed code.
2. Check Jenkins or GitHub Actions for new build triggers and monitor build status.
3. Jenkins steps:
 - a. Review Jenkins and check if a new build is running (lower left corner). The first build always takes more time, wait to the end, and get the results.
 - b. When complete successfully, it will be marked in blue next to the build number. If it is marked in red, there are errors.



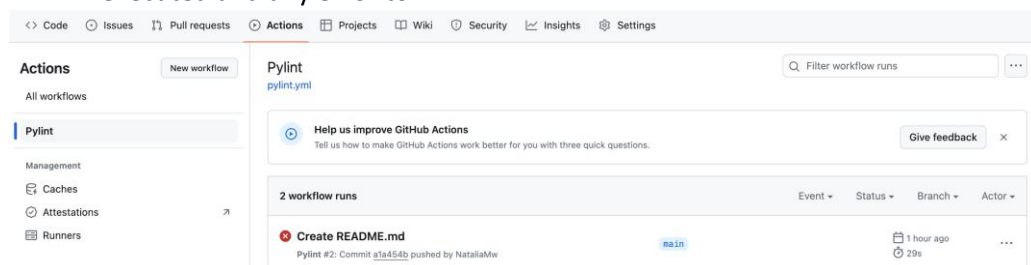
- c. From the side menu, select "Test Results Analyzer" to get charts about the tests performed and save those charts.
4. Github Actions steps:
 - a. Whenever a commit triggers an action, you will see an indicator in the form of a brown circle (in progress). This means the action is currently being executed.



- b. In case of error, it will appear as an X and you will be able to see details of the failure if you make click that icon:



- c. In the Action sections, you will also be able to see details of each workflow executed and any error to fix.



Build (3.9) failed 1 hour ago in 13s

- Set up job 1s
- Run actions/checkout@v4 1s
- Set up Python 3.9 8s
- Install dependencies 7s
- Analysing the code with pylint 1s

```

1 Run pylint $(git ls-files '*.py')
2 ===== Module calculator
3 calculator.py:2:0: C0114: Missing module docstring (missing-module-docstring)
4 calculator.py:4:0: C0116: Missing function or method docstring (missing-function-docstring)
5 calculator.py:8:0: C0116: Missing function or method docstring (missing-function-docstring)
6 calculator.py:12:0: C0116: Missing function or method docstring (missing-function-docstring)
7 calculator.py:16:0: C0116: Missing function or method docstring (missing-function-docstring)
8 ===== Module test_calculator
9 test_calculator.py:32:0: C0304: Final newline missing (missing-final-newline)
10 test_calculator.py:1:0: C0114: Missing module docstring (missing-module-docstring)
11 test_calculator.py:7:0: C0116: Missing function or method docstring (missing-function-docstring)
12 test_calculator.py:12:0: C0116: Missing function or method docstring (missing-function-docstring)
13 test_calculator.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
14 test_calculator.py:22:0: C0116: Missing function or method docstring (missing-function-docstring)
15 test_calculator.py:26:0: C0116: Missing function or method docstring (missing-function-docstring)

```

This branch is 1 commit ahead of, 145 commits behind master.

Nmwaiyin refactor: combine the complete execution with the individual n... a11be79 · 2 months ago 1,804 Commits

- d. In case of success, you will see a green circle or check alongside the commit.

Development

- Correct the necessary tests.
- Check that are enough test for each requirement
- Upload changes to repository
- Jenkins - Review Jenkins report (save charts)
- Github Action – Linting/tests report (before and after)
- Exported report from the test tool used

Deliverables:

1. Practice report with at least: cover, introduction, development, conclusions and recommendations, and references.
 - a. If it's a group project, include a roles table that highlights the responsibilities and roles of each team member.
 - b. Each screenshot must have the name of the student that made that part.
2. Answer these questions in the report:
 - a. What is continuous integration?
 - b. Why is it important and almost necessary?
 - c. In what type of projects is it recommended to apply it?
 - d. What would be the result of not doing the continuous integration of a project?
 - e. How does continuous integration align with modern agile software development practices?

Note: Do not include the questions, only the answer as sections.

3. In the development of the report, include screens of the practice process.
4. Include in the report the url of the repository where you did the practice.

Rubric

Description	Value
Testing code (in the repository)	
Jenkins/Github action configurations	10
Requirements fulfilled and quality of the code	15
Creation and correction of tests	10
Jenkins /Github action reports	20
Branches and correct commit management	5
Practice report	
Completions of requested sections in the report	10
Quality screens of the process	10
Quality answers to questions	10
Quality evidence of each part of the process	10
Total	100
Penalty per deliverables that missing or not uploaded	-30
Penalty per each requirement not fulfilled	-5
Individual penalty per missing evidence in taking part of the project.	-50
Penalty per missing repository link	-100
Penalty per missing names in the screenshot	-10

Late Submission Policy

Delay (§)	Penalty (Ω)
1 hour or less	loss of 10%
1 to 6 hours	loss of 20%
6 to 24 hours	loss of 30%
Over 24 hours:	loss of 100%

Links of interest:

YAML Basics: <https://learnxinyminutes.com/docs/yaml/>

Configuring Workflows: <https://docs.github.com/es/actions/using-workflows>

References

- [1] Amazon, «What is Continuous Integration?,» [En línea]. Available: <https://aws.amazon.com/devops/continuous-integration/>. [Último acceso: 13 Abril 2020].
- [2] Jenkins, «How to Install Jenkins on Windows,». [En línea]. Available: <https://www.jenkins.io/doc/book/installing/windows/>.
- [3] Jenkins, «Installing Jenkins on Linux,» [En línea]. Available: <https://www.jenkins.io/doc/book/installing/linux/>.

Gym Membership Management System

Description: Create a command-line application for managing gym memberships, calculating costs based on membership types, additional features per membership, and offering discounts for group memberships.

Assumptions can be made, but they should be specified in the document.

Requirements:

1. Membership Selection

- Display various gym membership plans with their benefits and costs (e.g., Basic, Premium, Family).
- Users can select a membership plan and proceed to customize it with additional features.

2. Additional Features per Membership

- Each membership plan allows users to add extra features (e.g., personal training sessions, group classes).
- Additional features have individual costs associated with them.

3. Cost Calculation

- **Base Membership Cost:** Determine the base cost for each selected membership plan.
- **Additional Features Cost:** Calculate the total cost of selected additional features.
- **Total Membership Cost:** Sum the base membership cost and the additional features cost to determine the total membership cost.

4. Discounts for Group Memberships

- If two or more members sign up for the same membership plan together, apply a 10% discount on the total membership cost.
- Display a message notifying users of potential savings when multiple members sign up together.

5. Special Offer Discounts

- Apply discounts based on the total cost of the selected membership plan and additional features:
 - If the total cost exceeds \$200, apply a discount of \$20.
 - If the total cost exceeds \$400, apply a discount of \$50.

6. Premium Membership Features

- Introduce premium membership features (e.g., access to exclusive gym facilities, specialized training programs) with higher costs.
- Apply an additional 15% surcharge to the total cost of memberships including premium features.

7. Membership Availability

- Validate that the selected membership plan and additional features are available.
- Display an error message if any selected option is unavailable and prompt users to select again.

8. User Confirmation

- Before finalizing the membership plan, display the selected membership, additional features, total cost, and applied discounts for user confirmation.

- Users can either confirm the membership plan or cancel and make changes to the selections.

9. Output

- Provide the total cost of the selected membership plan and additional features as a positive integer if the plan is confirmed and valid.
- Return a value of -1 if the input data is not valid or if the plan is canceled.

10. Error Handling

- Handle errors gracefully and provide clear error messages if users enter invalid input or if there are any calculation errors.
- Descriptive error messages should indicate the reason for the invalid input or cancellation.

11. Unit Testing

- Implement unit tests to ensure that each function works as expected and returns the correct output for different test cases.
- Ensure that validation and calculation functions are thoroughly tested to maintain accuracy and reliability.